CS3219 AY2627S1 Project Description

Expected Learning outcomes

- You should be able to apply the concepts learned in the lectures in a real-world context by designing a useful application that can be added to your portfolios.
- You should be able to extend your existing software engineering knowledge by:
  - developing a requirements specification (containing both functional and non-functional requirements),
  - developing an architectural design specification that satisfies the requirement specification, and
  - designing and developing an application following relevant design principles and patterns.
- You should be able to explore various architectures and design decisions in implementing the features of the product.
- You should be able to improve your communication skills (technical and non-technical) through collaborative group work, technical documentation writing, and presentations.

Project context

In this project, you will develop a medium-scale web application. The project involves designing and developing a peer-to-peer campus errand platform where students can request items to be collected from stores or facilities within the campus, and other students can fulfill (and deliver) these requests. The platform operates using a closed credit economy (i.e., credits cannot be purchased, withdrawn, or exchanged for money, and only circulate within the platform). During sign-up, each student receives an initial allocation of X credits (teams can assign any reasonable value to X) and earns additional credits by completing errands for other students.

A general description of the use of the platform is as follows:

> *A student who needs one or more items collected acts as a **requester**. The requester creates an account, logs in, and submits an errand request by selecting a pickup location and specifying the required delivery details. The system reserves one or more available credits when the request is created. Other students may act as **couriers** by viewing open requests and accepting one. Once a request is accepted, the assigned courier collects the items and updates the same in the system. After the items are delivered and the delivery is confirmed, the reserved credits are transferred from the requester to the courier. If the request is canceled or is not accepted within*

the permitted duration, it should expire or be canceled appropriately, and the reserved credits should be released.

Students may use the platform both to request errands and to fulfill requests submitted by others. You are encouraged to explore the requirements and architectural design of such an application without deviating from its overarching purpose.

Implementation Expectations

All mandatory features/functionality, behavior, and deployment related descriptions are labelled as Mx in the following.

M1. Responsive Design and User Experience

The web application should provide a responsive, intuitive user interface that works across desktop and mobile screen sizes and supports complete workflows for both requesters and couriers. You are encouraged to demonstrate strong product thinking in screen organization, navigation, user feedback, and handling invalid or unsuccessful actions.

2. Mandatory Microservices

The application should follow a microservice-based architecture and include the following mandatory services.

M2. User Service

The User Service should manage user registration, authentication, profile information, and a user's ability to participate as both a requester and a courier.

M3. Supplier Service

The Supplier Service should maintain information about the stores (e.g., CoffeeBean@Com3), facilities (e.g., Printers@PCCommons), and/or locations (e.g., PGP Foyer) from which users may request errands. It should support CRUD on campus suppliers/vendors/landmarks. Use the initial data provided in the template repository to seed your application. You are expected to add more to ensure a rich listing.

M4. Order Service

The Order Service should manage the complete lifecycle of an errand request. This includes creating a request, listing open requests, accepting a request, courier assignment, marking an item/request as picked up, completing or canceling a request, and handling requests that expire before being accepted. You are expected to handle corner cases such as ensuring only one courier can be assigned to an order and that the requester cannot accept their own request.

M5. Credit Service

The Credit Service manages the closed credit economy of the platform. Users receive an initial credit allocation when they register. When an errand request is created, the system should reserve the required credits from the requester's available balance and transfer them to the courier upon successful completion of the errand. Credits should be released if the request is canceled or expires.

Note: You are expected to add additional services where appropriate.

M6. Asynchronous/Event-Driven Communication

The application must include meaningful asynchronous or event-driven (or both) workflows. You should identify interactions that do not require immediate synchronous communication and are suitable for asynchronous handling. The selected workflow(s) should support a genuine product or system requirement.

M7. Containerization

All application services and supporting components should be containerized (e.g., using Docker) and deployed locally (i.e., on your computers) or on the cloud. Note: While deployment on cloud is strongly encouraged, the scope of must-have deployment is limited to local deployment.

Assumptions and Scope Boundaries

The following assumptions may be made while designing the application:

- Suppliers and valid campus locations may be predefined in the system.
- Payment for the items collected from a supplier is outside the scope of the platform.
- Platform credits represent compensation for completing an errand and are not used to pay the vendor.
- Supplier menus, product catalogs, inventory, and checkout are outside the project scope.
- Courier (live) location-tracking during transit is outside the project scope.

Students may make additional reasonable assumptions where requirements are not explicitly specified. Such assumptions should be documented and should not alter the overarching purpose of the application.

Note: When in doubt, get your assumptions clarified by talking to your mentor or the course coordinator.

Nice-to-Have Features

The following are the nice-to-have (N2H) categories, labelled Nx, identified by the teaching team for this project. They are neither exhaustive nor prescriptive, and your team is free to design, adapt, or extend them based on your interpretation of the project.

You are not required to limit yourselves strictly to the categories or items listed below. If your team identifies features that are more aligned with the objectives of the project, you may propose and implement them instead. In such cases, discuss the proposed N2H features with your mentor and lecturer before implementation to ensure that they remain within the scope and purpose of the project, and are of appropriate complexity.

While there is no fixed limit on the number of N2H features a team may implement, every team member must contribute to the implementation of at least one N2H feature worth of effort. Teams should distribute the work reasonably equally among team members and ensure that the overall workload remains manageable within the project timeline.

When presenting an N2H feature, your team should explain its relevance to the platform, the problem it addresses, and the technical design and implementation choices involved. The objective is not simply to implement a larger number of features, but to use the selected N2Hs to demonstrate your team's strengths, product thinking, and ability to independently explore relevant technologies and design approaches.

Some of the concepts required to implement the must-have or N2H features may not be explicitly covered in lectures or tutorials. Teams are therefore encouraged to conduct independent research where required.

N1: Application Enhancements

These features extend the functionality and user experience of the core application. Possible enhancements are as follows:

- **Requester enhancements**
  - Scheduled errands
  - Order history + Reordering errand flow
  - Bookmarking delivery and pickup locations
- **Courier enhancements**
  - Route visualization and navigation support
- **Communication**
  - Real-time chat, voice/video call
  - Real-time status update push notifications to user
- **Admin Dashboard**
  - Live and past orders listing
  - Advanced user management flows, e.g., user suspension, account review and reinstatement based on evidence drawn from system logs
  - Failed-order and dispute management
  - User/courier login audit logs
  - Application-level, user-level, task-level analytics

N2: AI Features and External Integrations

These features introduce intelligent capabilities or integrate the application with external systems and services. Possible enhancements include:

- **Natural-language or voice-based errand creation:** Allow requesters to describe an errand using natural language or voice, with the system extracting the relevant details and converting them into a structured errand request.
- **Real-time chat translation:** Translate messages between requesters and couriers in real time to support communication across the multilingual NUS student community.
- **AI-assisted proof-of-delivery verification:** Create a proof of delivery flow where AI analyses the images submitted by the courier to verify whether an errand appears to have been successfully completed.
- **RAG-based FAQ/support:** A RAG-based chatbot to help with FAQ and support (with reasonable supporting documentation created, system logging, etc) to answer user's questions.
- **Task-credits-estimation assistant**: Analytics based task-credits estimation assistant to recommend credits allocation during new tasks creation, or verification of credit assignment when accepting an errand.
- **Content moderation on in-app communication:** Flag/mute abusive, unsafe communication between requester and courier before being delivered.
- **Smart request-to-courier ranking/matching:** Use an LLM/ranking model to match requests and explain to the courier why the request is a good match.

N3: Testing, Reliability, and Observability

These features improve confidence in the correctness, reliability, and operational behavior of the application. Possible enhancements include:

- comprehensive automated testing beyond the unit- and integration-testing, including end-to-end testing, load, stress, or concurrency testing, demonstrated with appropriate evidence
- centralized logging
- automated health monitoring and alerts via monitoring dashboards
- distributed message/communication tracing
- enhanced handling and recovery of failed asynchronous messages
- responsible handling of data sent to third-party services and LLMs

N4: Cloud deployment and DevOps

These features extend the mandatory containerized setup and demonstrate more advanced deployment and software delivery practices. Possible enhancements include:

- establishing CI/CD pipelines
- deployment to a cloud platform
- Kubernetes deployment and automated scaling, demonstrated with evidence
- automated build, test, and deployment workflows
- infrastructure-as-code; secrets and configuration management
- production-oriented deployment and monitoring setup

Organizing work within your Project Team

- Each project team will have 4 to 5 members. Teams of less than four or more than five members are disallowed.
    - For better collaboration, it is strongly encouraged to form teams within the same tutorial.
- Allocate the responsibility of completing the product features (i.e., must-haves and nice-to-haves) and the remaining project tasks (i.e., documentation, demo, and presentations preparation) roughly equally among the team members.
- Keep a note of the individual contributions. This will be factored in while grading the project components and marks for individual members may differ.
    - You are required to declare your role allocation and responsibilities in the final presentation.
    - Additionally, we will use peer reviews and mentors' feedback to gather information about the individual contributions and team dynamics. The teaching team will intervene if necessary.
- A mentor will be allocated to each project team.

Reuse policy and declaration

- Each group is expected to sign a declaration (see Appendix 1) that the project is bona fide work of the team.
- If any code is reused, or any reference is made to an existing design or documentation, acknowledge the source(s).
- Attach this declaration to your final submission (the presentation slides) before submitting it on Canvas.
- You are expected to read through the course's AI usage policy in Appendix 2.

Project Code Management

You will be given a template repository, which **should be forked** to your team organization.

- The team organization name should be named as follows: AY2627S1-CS3219-P<groupnumber> (e.g., AY2627S1-CS3219-P1, AY2627S1-CS3219-P11, etc.,).

- You should use a single-repository structure with one folder per microservice.
- You should create a PR to the template repository from your team organization's repository to facilitate code review and contribution tracking, where necessary.

Project Timeline

To facilitate incremental work on the project, we have 4 graded milestones during the semester. A summary is provided below.

Note: All graded milestones are in-person/face-to-face assessments. All team members are expected to be present irrespective of who contributed to the deliverable.

Milestones	Deliverables	Weightage (%)	Submission Format and Deadline
D1	Project backlog and prototype presentation (defining project scope)	5	1/ F2F presentation (20 min) to the lecturer + TA/mentor, followed by QA (10 min) before the end of Week 5 (Between Sep 9-12, 2026); 2/ Backlog document submission on Canvas by Sep 17, 2026 at 23:59
D2	First graded progress check	5	1/ F2F presentation (including demonstration) to mentor in Week 7.
D3	Second graded progress check	10	1/ F2F presentation (including demonstration) to mentor in Week 10
D4	Full project demonstration with slides.	20	1/ Slides to be submitted and product final commit by Nov 11, 2026, 10:00 hrs. 2/ F2F presentation with demo (30 min), followed by QA (10 min) to graders before the end of Week 13 (between Nov 11- 14, 2026). Demo slot selection will open at least a week before.

Milestone D1: Project Outline

Weightage: 5 points; Presentations: Week 5

In this milestone, you will develop:

- The project requirements in the form of a product backlog.
- Wireframes or prototype user interfaces for the major product workflows, showcasing the UI's responsiveness (i.e., compatibility with laptop/computer screens as well as handphone screens))

Tasks

1. Product Backlog

Refer to the project description and mandatory features as the high-level capabilities expected of the product.

a. a. Analyze each feature's capabilities, think through them, and write the requirements for each feature; refine each high-level requirement through at least two levels of refinement/decomposition (E.g., FR → FR1.1 → FR1.1.1; FR → FR1.1 → FR1.1.2, etc.). This milestone expects you to apply requirements analysis and requirements specification concepts.

b. b. The requirements should be categorized (e.g., FR/NFR), indexed, and prioritized. You can refer to the example screenshot in the lecture notes.

c. c. Refer to the nice-to-have (N2H) categories, N1 to N4. Decide which N2H your team wishes to explore and develop in the project. Capture the key requirements for these identified nice-to-haves and document them briefly (in about 2-3 lines each) at the end of the product backlog. You should revise, refine, and elaborate on them during Milestone D3.

2. UI Wireframes

Envision how users will interact with the application's major features.

You may:

- develop a coded prototype;
- draw clear wireframe diagrams; or
- create an interactive UI prototype.

The wireframes should cover the main workflows for requesters and couriers and demonstrate how the application adapts to desktop and mobile screen sizes.

Evaluation

Your team should be prepared to present:

- the product backlog, including the identified FRs and NFRs

- the responsive UI wireframes

We will open a 30-minute slot in Week 5 for each team to present your product backlog to the teaching team (lecturer + TA/mentor). You are encouraged to prepare a simple slide deck for your presentation.

Note: Beautification of slides carries no marks! So, don't spend time on it. Avoid reading from a script while presenting your work.

Milestone D2: Progress Check – 1

Weightage: 5 points; When: Week 7

Milestone D2 is an early-project progress check. The goal is to ensure that your team has made concrete design and implementation decisions for the foundational services and that you are on track to deliver a complete system.

At this stage, you are encouraged to have a near-complete implementation of at least one of User Service or Supplier Service and made significant progress on the other. Your team should demonstrate clear design decisions, evidence of implementation, and an understanding of how these services will support the later Order and Credit workflows.

Tasks for this Milestone

Your team is expected to:

- design and develop the User Service, including authentication and authorization, RBAC, profile management, etc.,
- design and develop the Supplier Service to support CRUD operations
- implement a basic responsive supplier-listing page
- demonstrate interactions between the frontend and the foundational services using a containerized deployment

Design & Implementation Decisions to be Checked

1. User Service

Your team should be prepared to explain and demonstrate:

- choice of database for the User Service, including user data model & schema design
- user management and authentication – implementation decisions and approach
- user role management: requester/courier/admin, including role-toggle functionality between requester and courier
- integration (or plan thereof) of the User Service with other services

2. Supplier Service

Your team should be prepared to explain and demonstrate:

- choice of database for the Supplier Service, supplier data model and how supplier data is queried
- supported search and filtering capabilities

3. Supplier-Listing Page on the Responsive UI

Your team should be prepared to demonstrate:

- how the page behaves across desktop and mobile screen sizes
- how the frontend retrieves data from the Supplier Service

Milestone D3: Progress Check – 2

Weightage: 10 points; When: Week 10

Milestone D3 is a mid-project progress check. The goal is to ensure that your team has made concrete design and implementation decisions for the core product workflow and that you are on track to deliver a complete, integrated system.
At this stage, the Order Service, Credit Service, asynchronous workflow, and containerized setup should be sufficiently advanced to demonstrate feasibility and integration. The implementation need not be fully complete, but the major design decisions should be finalized.

Tasks for This Milestone

Your team is expected to:

- design and develop the Order and Credit Services and integrate them with the other services (Note: your integration need not be complete at this stage, but there should be evidence of integration with the other services)
- implement at least one meaningful asynchronous or event-driven workflow
- containerize the services and supporting infrastructure
- finalize the N2H features to be implemented and the approach to implementing them

Design & Implementation Decisions to be Checked

1. Order Service

Your team should be prepared to explain and demonstrate:

- database choice, order data model, and schema design
- the complete order lifecycle along with valid state transitions
- how the system handles corner cases like cancellation, expiry, etc
- the integration of the Order Service with other services

2. Credit Service

Your team should be prepared to explain and demonstrate:

- how the initial credit allocation is handled
- credit reservation upon order placement and release and transfer upon order completion or cancellation
- how atomicity is achieved for credit-related operations
- the integration of credit service with other services

3. Asynchronous and Event-Driven Workflow

Your team should be prepared to explain and demonstrate:

- the chosen workflow(s) for asynchronous processing and the rationale behind it, including whether event ordering matters
- the selected technology for asynchronous communication and the reasons for its choice
- the event schema and the information it conveys
- how duplicate events and failures are managed
- which parts of the system are eventually consistent due to the current implementation
- any observability tools in place to monitor performance and detect anomalies

The chosen asynchronous flow should represent a meaningful part of the product rather than being introduced solely to demonstrate the use of a message broker.

4. Service Containerization

Your team should be prepared to explain and demonstrate:

- the Dockerfile strategy for each service, databases, and supporting infrastructure (such as messaging), and the use of Docker Compose (if any)
- dependency and configuration management
- service communication
- any deployment or CI/CD considerations that have influenced the design.

The team should demonstrate that the implemented services can be built and started using the containerized setup.

5. Nice-to-Have Features

Your team should be prepared to present:

- the selected N2H features, and their relevance to the product and/or to one another
- the proposed implementation approaches and their integration with the system
- the allocation of responsibilities among team members

Teams should prioritize meaningful, well-integrated N2H over a larger number of incomplete additions.

Milestone D4: Final Submission & Presentation

Weightage: 20 points; Presentations: Week 13

The objective of this milestone is to showcase your completed FoC system, explain your design and implementation decisions and reflect on your team's contributions.

Deliverables

1. Demo

- A well-prepared live product demonstration showing your system in action.
- Use prepared scenarios and test data to highlight the main workflows (e.g., requesting an order, accepting an order, transferring credits).
- Cover the core services and N2H you have implemented.
- If deployed online, demonstrate using the deployment link. Otherwise, demonstrate the system running locally in containers. Declare which mode you are using during the demonstration.

2. Presentation slides (See submission instructions)

Your slides should be concise (~15-20 slides) and include, but not limited to:

- Team introduction → team name, role/task, N2H allocation per member.
- Tech stack → languages, frameworks, databases, tools.
- Architecture and/or deployment diagram → microservices layout, data flow, key integrations.
- The following key decisions:
    - Database choice for the key services and the patterns used.
    - Implementation & deployment strategy for services.
    - Nice-to-Have feature(s): highlight those where specific technologies or approaches were used, which can distinguish your work from others
    - Any other decision that you made when designing/implementing FoC
- Repository link → GitHub repo with a clear README.
- Deployment link (if available).
- Supporting documentation link (if prepared) → e.g., runbooks, API docs,
- design docs.
- Signed declaration (See Appendix 1)

Submission Instructions

- Submission date**: November 11, 2026, 10:00 AM** (including final code commit)
- Upload your presentation slides to CANVAS under FoC Presentation.

- Use the filename format: ProjectGroup<number>.pptx (e.g., ProjectGroup5.pptx)
- If you have additional files (e.g., documentation, readme, runbook, diagrams, secrets that are necessary to run the application), submit them to the separate assignment, "FoC Other Documents" as a separate zip file.
    - Create a single ZIP file containing all your additional files.
    - ZIP filename format: ProjectGroup<number>-additionalfiles.zip (e.g., ProjectGroup5-additionalfiles.zip).

Evaluation Criteria

- Clarity → slides are well-structured, concise, and easy to follow.
- Completeness → demo + slides together cover architecture, tech choices, team
- contributions, and key decisions.
- Technical depth → rationale for decisions is explained; trade-offs and integration considerations addressed.
- Team contribution → roles/tasks are clear; each member demonstrates ownership.
- Professionalism → delivery, flow, and timing of presentation. Avoid reading from scripts.
- Product demonstration → Focus on showing the system in action and explaining why you made your choices rather than covering every minor detail.
- Test data and storyline → Seed sample users and suppliers; develop scenarios ahead of time so the flow looks smooth.
    - Use meaningful names for users and have meaningful suppliers populated in the database.
    - Use the demo to tell a story (e.g., "A user wants his prints to be brought from SOC to PGP … "). Rehearse your demo to avoid surprises.

Appendix 1: Declaration

This declaration should be printed, completed, scanned, and attached to the slide deck submitted on Canvas as the last slide

We, the undersigned, declare that:

1. The work submitted as part of this project is our own and has been done in collaboration with the members of our group and no external parties.
2. We have not used or copied any other person's work without proper acknowledgment.
3. Where we have consulted the work of others, we have cited the source in the text and included the appropriate references.
4. We understand that plagiarism is a serious academic offense and may result in penalties, including failing the project or course.
5. We have read the NUS plagiarism policy and the Usage of Generative AI.

Group Member Signatures:

Full Name (as in Edu Rec)	**Signature**	**Date**

Appendix 2: AI Usage Policy for CS3219

The following applies to all your work under the team project in this course.

What's Allowed

You may use AI tools* for:

- Requirements work: discovering, interpreting, formatting, specific style writing
- Writing implementation code (e.g., functions, classes, unit tests) once requirements and architecture are finalized by you.
- Boilerplate generation (e.g., config, scaffolding, repetitive glue code).
- Debugging assistance (e.g., error explanations, test suggestions).
- Refactoring and documentation improvements (e.g., docstrings, comments).
- Learning support (e.g., "explain this algorithm").

All allowed uses require explicit citation (see citation point below).

** AI tools: Systems that generate or transform text/code (e.g., ChatGPT, GitHub Copilot, etc.,).*

What's Not Allowed

You must not use AI tools for:

- Requirements work: wholesale outsourcing of requirements elicitation to AI tools, prioritizing project requirements; consolidating backlog, sprint planning.
- Architecture & design: proposing/changing system architecture, component boundaries, selecting design patterns, deciding data schemas, defining interfaces, or making performance/security trade-offs.
- Decision rationales: drafting your trade-off analyses, risk mentions, or justification mentions.

Your Responsibilities

- Accountable: You remain fully responsible for understanding and validating any AI-assisted code.
- Quality & compliance: Ensure outputs meet assignment specs, performance or security guidelines, and licensing constraints.
- Privacy: Do not paste proprietary, personal, or assessment content into tools that store prompts.

Required Citation & Disclosure

Every submission using AI must include:

- Source & mode: which tool(s), how it was used (generate/refactor/debug/explain).
- Prompts: the exact prompts + key responses.
- Location(s):
    - place the attribution comment at the top of each AI-influenced file (see below) as a comment,

- a consolidated disclosure in your project README as the last section. Add a link to the README in the slide deck
- a usage log file in the project repository

Example: Short File-Header Attribution (in each affected file)

AI Assistance Disclosure:
Tool: ChatGPT (model: GPT-5.6 Luna Light), date: 2026-08-11
Scope: Generated initial implementation of modules X and Y; suggested test cases for Z.
Author review: I validated correctness, edited for style, and added boundary checks.

Example: Project-Level Disclosure

(put in README and add link to the README in submission slide deck)
AI Use Summary
Tools: ChatGPT (GPT-5.6 Luna Light), GitHub Copilot
Prohibited phases avoided: requirements elicitation; architecture/design decisions.
Used for:

- Generating boilerplate for Express server and Jest config.
- Getting suggestions for refactoring; for data parsing function; I retained A, rejected B (explanation below).
- Generating unit tests for edge cases (I added two additional tests). Verification: All AI outputs reviewed, edited, and tested by the authors. Prompts/Key Exchanges: See /ai/usage-log.md at the end of this segment.

Example: Logging

- Maintain a log, /ai/usage-log.md, in the repository with timestamps, prompts, and usage scenarios.
- Mark any pasted AI code blocks with comments like:
  // AI-generated (edited by <name>).
- Add all your SKILLS.md/AGENTS.md to the repository before the final submission.

Academic Integrity & Licensing

- You must ensure generated code does not introduce incompatible licenses or plagiarized content. If the tool cites a source, credit it.
- If unsure about licensing, rewrite in your own words or implement from specifications.

Evaluation & Penalties

- Missing or misleading disclosure may reduce the assignment grade or be treated as an academic integrity violation.

- Using AI in prohibited phases (requirements/architecture/design) will incur penalties up to receiving a zero on the project.

Team Agreement

- Teams must agree on AI usage norms and maintain a shared /ai/usage-log.md.
- Each member remains accountable for understanding the entire codebase.

Quick Checklist (before you submit your work)

- Requirements and architecture not generated wholesale by AI.
- AI is used only for implementation/debugging/refactoring/docs.
- All AI-influenced files have header attributions.
- README/report includes the project-level AI use summary.
- Prompts and key outputs archived in /ai/usage-log.md.
- All AI outputs are reviewed, tested, and verified by the authors.

For further reading and reference, here is the NUS AI Policy:
https://libguides.nus.edu.sg/new2nus/ai_guidelines_infographics